# Direct Methods in Linear Algebra - Part 3

Alexandra A. Gomes

February 18, 2026

# Example: Why Pivoting Matters

Consider

$$A = \begin{pmatrix} 10^{-4} & 1 \\ 1 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Without pivoting:

$$m_{21} = \frac{a_{21}}{a_{11}} = \frac{1}{10^{-4}} = 10^4.$$

$$r_2 \leftarrow r_2 - m_{21} r_1 : \quad (1, 1 \mid 2) - 10^4 (10^{-4}, 1 \mid 1) = (0, -9999 \mid -9998).$$

**Problem:** huge multiplier $\Rightarrow$ strong amplification of rounding errors

# Partial Pivoting

The pivot in step $k$ is the element in column $k$, below the diagonal, with the largest absolute value:

1. determine the smallest $p \geq k$ such that

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|;$$

2. $r_k \leftrightarrow r_p$.

# Same Example with Partial Pivoting

Partial pivoting at $k = 1$:

$$\max\{|10^{-4}|, |1|\} = 1 \Rightarrow p = 2, \qquad r_1 \leftrightarrow r_2.$$

$$\left(\begin{array}{cc|c} 1 & 1 & 2 \\ 10^{-4} & 1 & 1 \end{array}\right)$$

Do elimination:

$$m_{21} = \frac{10^{-4}}{1} = 10^{-4}, \qquad r_2 \leftarrow r_2 - 10^{-4} r_1.$$

$$(10^{-4}, 1 \mid 1) - 10^{-4}(1, 1 \mid 2) = (0,\ 1 - 10^{-4} \mid 1 - 2 \cdot 10^{-4}).$$

**Takeaway:** choosing the largest entry in the column as pivot keeps multipliers small, improving numerical stability.

# Toy Rounding Demo (3 Significant Digits)

With operations rounded to 3 significant digits.

**No pivoting:** $m_{21} = 10^4$. Update the RHS:

$$b_2^{(2)} = b_2 - m_{21} b_1 = 2 - 10^4 \cdot 1.$$

With 3 sig digits:

$$2.00 - 1.00 \times 10^4 \approx -1.00 \times 10^4.$$

**With partial pivoting:** swap rows first, then $m_{21} = 10^{-4}$.

$$b_2^{(2)} = 1 - 10^{-4} \cdot 2 = 0.9998 \approx 1.00$$

(no catastrophic loss of information).

**Takeaway:** pivoting avoids subtractions like $2 - 10000$ that destroy low-order digits and propagate error through back-substitution.

# Scaled Partial Pivoting

**Scaled Partial Pivoting**:

1. for each row, define $s_i$ such that

$$s_i = \max_{1 \leq j \leq n} |a_{ij}|;$$

   we have a scale vector $\mathbf{s} = [s_1 \quad \ldots \quad s_n]$;

2. for the first column, choose as pivot

$$\frac{a_{p1}}{s_p} = \max_{1 \leq k \leq n} \frac{|a_{k1}|}{s_k}$$

   and $r_1 \leftrightarrow r_p$.

3. do the same for the other columns: select the smallest $p \geq i$ such that

$$\frac{a_{pi}}{s_p} = \max_{i \leq k \leq n} \frac{|a_{ki}|}{s_k}$$

   and $r_i \leftrightarrow r_p$ (if $i \neq k$).

# Scaled Partial Pivoting: Example

$$A = \begin{pmatrix} 1000 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 3 & 4 \end{pmatrix}$$

**Step 1: compute row scales** $s_i = \max_j |a_{ij}|$**:**

$$s_1 = 1000, \qquad s_2 = 2, \qquad s_3 = 4.$$

**Step 2: compute scaled ratios in column 1:**

$$\frac{|a_{11}|}{s_1} = \frac{1000}{1000} = 1, \qquad \frac{|a_{21}|}{s_2} = \frac{2}{2} = 1, \qquad \frac{|a_{31}|}{s_3} = \frac{1}{4} = 0.25.$$

**Pivot choice (smallest index attaining the max):**

$$\max\{1, 1, 0.25\} = 1 \;\Rightarrow\; p = 1.$$

**Scaled partial pivoting keeps** $r_1$ **as the pivot row, as does partial pivoting.**

# Scaled Partial Pivoting: Example

With pivot $a_{11} = 1000$, eliminate entries below it:

$$m_{21} = \frac{2}{1000} = 0.002, \qquad m_{31} = \frac{1}{1000} = 0.001.$$

Update rows ($r_i \leftarrow r_i - m_{i1} r_1$):

$$r_2 : \ (2, 2, 2) - 0.002(1000, 1, 1) = (0, \ 1.998, \ 1.998),$$

$$r_3 : \ (1, 3, 4) - 0.001(1000, 1, 1) = (0, \ 2.999, \ 3.999).$$

**Scaled pivot choice in column 2 among rows 2–3:** Keep the same scales from the start: $s_2 = 2, s_3 = 4$.

Compute scaled ratios in column 2:

$$\frac{|a_{22}^{(2)}|}{s_2} = \frac{1.998}{2} = 0.999, \qquad \frac{|a_{32}^{(2)}|}{s_3} = \frac{2.999}{4} \approx 0.750.$$

# Scaled Partial Pivoting: Example

**Pivot choice (smallest index attaining the max):** pick row 2 as pivot (no swap), pivot $a_{22}^{(2)} \approx 1.998$.

**Then eliminate the** $(3, 2)$ **entry:**

$$m_{32} = \frac{2.999}{1.998} \approx 1.50, \qquad r_3 \leftarrow r_3 - m_{32} r_2.$$

**Note that ordinary partial pivoting would have chosen to swap rows 2 and 3** to make 2.999 the pivot.

**Takeway**: ordinary pivoting picks the largest absolute entry; scaled pivoting picks the entry that is largest relative to what is typical in that row (the row's scale).

# No Pivoting

Are there matrices that do not require pivoting? Yes:

- symmetric positive definite (SPD) matrices;
- diagonally-dominant matrices;

# Symmetric Positive Definite Matrices

- $A$ is symmetric: $A = A^T$;
- $A$ is SPD: all pivots are positive;

GE is stable for an SPD matrix: no pivoting is needed.

# Cholesky Decomposition for $A$ SPD

$A$ is SPD with $A = LU$, with $L$ unit lower triangular and

$$U = \begin{bmatrix} u_{11} & & & \\ & u_{22} & & \\ & & \ddots & \\ & & & u_{nn} \end{bmatrix} \begin{bmatrix} 1 & \frac{u_{12}}{u_{11}} & \cdots & \frac{u_{1n}}{u_{11}} \\ & 1 & \cdots & \frac{u_{2n}}{u_{22}} \\ & & \ddots & \vdots \\ & & & 1 \end{bmatrix} = D\tilde{U}.$$

Then: $LD\tilde{U} = \tilde{U}^T DL^T$. Since this decomposition is unique: $\tilde{U}^T = L$. Thus,

$$A = LDL^T.$$

# Cholesky Decomposition for $A$ SPD

From $A = LDL^T$ and $u_{ii} > 0$,

$$D = D^{1/2} D^{1/2}$$

with $D^{1/2} = \text{diag}(\sqrt{u_{11}}, \ldots, \sqrt{u_{nn}})$. Then,

$$A = (LD^{1/2})(D^{1/2}L^T) = L'L'^T,$$

with $L' = LD^{1/2}$, lower triangular, with positive diagonal elements.

1. half the memory space;
2. complexity: imposing $A = L'L'^T$ element by element results in

$$\frac{1}{3}n^3 + \mathcal{O}(n^2).$$

3. in Matlab: $R = \texttt{chol(A)}$, with $R = L'^T$.

# Cholesky Decomposition: Example

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{pmatrix}.$$

It is SPD (all leading principal minors are positive):

$$\Delta_1 = 4 > 0, \qquad \Delta_2 = \det\begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix} = 16 > 0, \qquad \Delta_3 = \det(A) > 0.$$

We seek a lower triangular $L$ with positive diagonal such that

$$A = LL^T, \qquad L = \begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix}.$$

# Step 1: First Column of $L$

From $A = LL^T$, compare entries.

Diagonal entry $(1,1)$:

$$\ell_{11}^2 = a_{11} = 4 \quad \Rightarrow \quad \ell_{11} = 2.$$

Off-diagonal entries in column 1:

$$a_{21} = \ell_{21}\ell_{11} = 2 \quad \Rightarrow \quad \ell_{21} = \frac{2}{2} = 1,$$

$$a_{31} = \ell_{31}\ell_{11} = 2 \quad \Rightarrow \quad \ell_{31} = \frac{2}{2} = 1.$$

So far,

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & \ell_{22} & 0 \\ 1 & \ell_{32} & \ell_{33} \end{pmatrix}.$$

# Step 2: Second Column of $L$

Diagonal entry $(2, 2)$:

$$a_{22} = \ell_{21}^2 + \ell_{22}^2 \quad \Rightarrow \quad 5 = 1^2 + \ell_{22}^2 \quad \Rightarrow \quad \ell_{22} = 2.$$

Off-diagonal entry $(3, 2)$:

$$a_{32} = \ell_{31}\ell_{21} + \ell_{32}\ell_{22}.$$

Plug in known values $a_{32} = 1$, $\ell_{31} = 1$, $\ell_{21} = 1$, $\ell_{22} = 2$:

$$1 = (1)(1) + \ell_{32}(2) \quad \Rightarrow \quad \ell_{32} = 0.$$

Now

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \ell_{33} \end{pmatrix}.$$

# Step 3: Third Diagonal Entry

Diagonal entry $(3, 3)$:

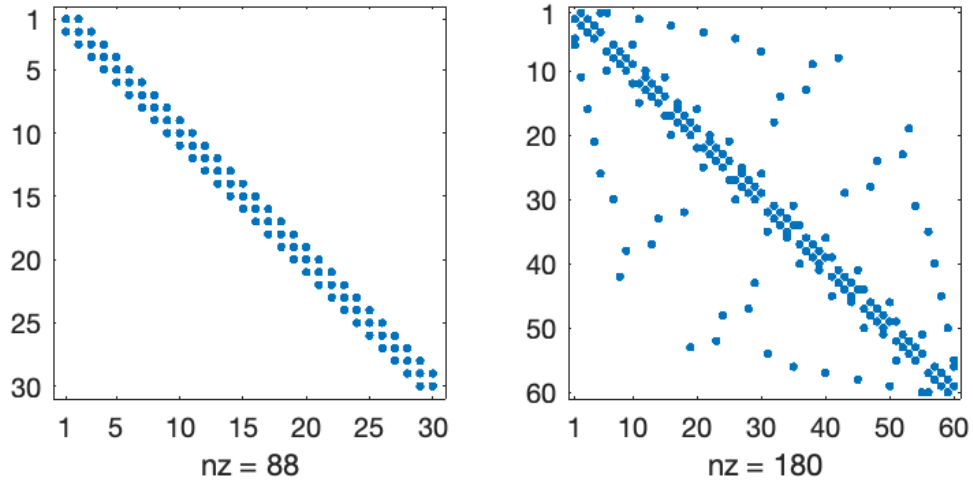
$$a_{33} = \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 \quad \Rightarrow \quad 3 = 1^2 + 0^2 + \ell_{33}^2 \quad \Rightarrow \quad \ell_{33} = \sqrt{2}.$$

Therefore,

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{pmatrix}, \qquad A = LL^T.$$

# Sparse Matrices

- in practice, matrices often have very few non-zero elements: sparse matrices;
- banded matrices are a type of of sparse matrices;
- substantial gains in storage and flop:
  - do not store the zeros;
  - do not multiply by the zeros;

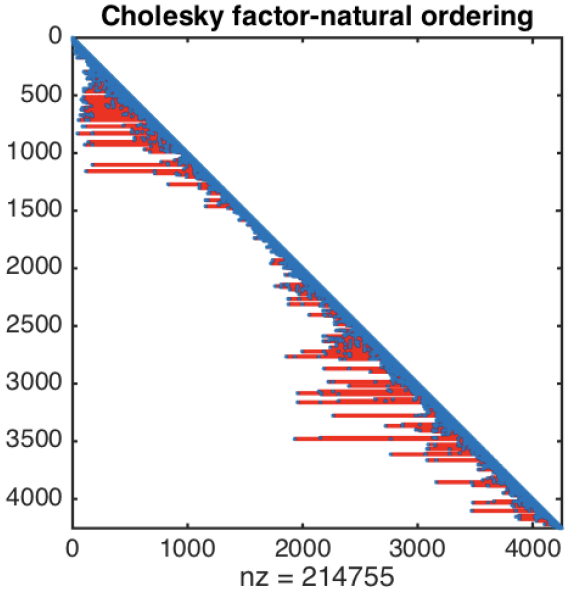


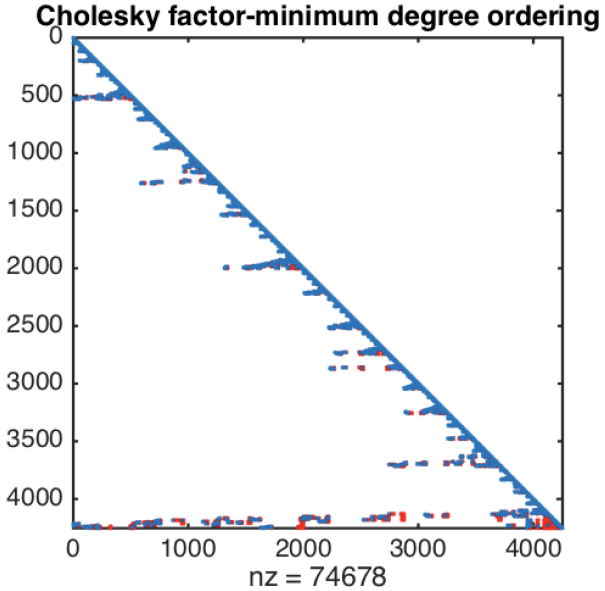
# Sparse Matrices: Storage

- dense matrices are stored as a $n^2$ array;
- the simplest storage format for sparse matrix with $m$ non-zero entries is the triplet form:
  - three arrays **i**, **j**, and **v** of size $m$;
  - **i**, **j** hold integers indicating the row/column location of entries;
  - **v** holds the floating-point entries;
- more sophisticated formats are common in high performance computing environments;

# Sparse Matrices: Operations

- take advantage of sparsity when coding and using libraries;
- $LU$ decomposition of sparse matrices: fill-in, i.e., an original zero is overwritten by a non-zero;
- $LU$ decomposition of banded matrices:
  - no pivoting and no zeros in the band: no fill-in;
  - row permutations: fill-in;
- iterative methods: avoid factorization altogether (or use an incomplete factorization preconditioner that limits fill).



# Sparse Matrices: Practice

- $LU$ (and its symmetric cousins $LL^T$, $LDL^T$), $Q\Lambda Q^T$, $QR$, and $U\Sigma V^T$ routines are available in most numerical computing libraries;
- these employ sophisticated permutations strategies, not just for improving stability, but for reducing the fill-in due to operations;
- in Matlab:
  - `S = sparse(A)` converts a full matrix to a sparse one;
  - `S = sparse(m, n)` creates a sparse $m \times n$ matrix;
  - `spy(S)` displays the sparsity pattern of a sparse matrix;
  - `S = sparse(i, j, v)` converts the triplet representation to the efficient internal representation (CSR);
  - all the usual factorization routines (`lu`, `chol`, etc.) work and return sparse factors but fill-in generally occurs;